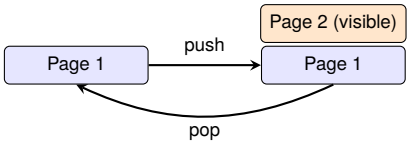
Flutter routes

Worked examples: two screens, an employee salary app, a restaurant menu

Key ideas

- A **screen**: usually its own `Scaffold`, `AppBar`, body.
- A **route**: an entry managed by a `Navigator`; it shows a screen.
- `MaterialApp(home: ...)` creates the first route and a navigator.
- The navigator keeps a **stack** of routes: last in, first out. The top one is visible.



The screen below stays mounted: its fields keep their text.

Push and pop

```
// Page 1: open Page 2
Navigator.of(context).push<void>(
  MaterialPageRoute<void>(
    builder: (context) => const Page2(),
  ),
);
// Page 2: go back
Navigator.of(context).pop();
```

`push(route)`	adds a route on top and shows it
`pop()`	removes the top route, reveals the one below
`MaterialPageRoute`	builds the screen; Material transition
`<void>`	this route returns no result

Page 2 also gets an automatic back arrow. A navigation callback alone does not need a `StatefulWidget`.

To go back, `pop`. Pushing Page 1 again adds a *new* route: the stack grows at every "return".

BuildContext

- A `BuildContext` is a widget's **location** in the tree, used to look *up*: `Navigator.of`, `Theme.of`, `MediaQuery.of`, `ModalRoute.of`.
- It does not contain the data or the arguments.
- The context of `MyApp.build` is *above* the `MaterialApp`: it cannot find its navigator. Navigate from a screen.
- After an `await`, check `mounted` before using it.

Icons in navigation buttons

```
ElevatedButton(onPressed: open,
  child: const Icon(Icons.navigate_next,
                    size: 50))
ElevatedButton.icon(        // with a label
  onPressed: openPage2,
  icon: const Icon(Icons.navigate_next),
  label: const Text('Show salary'))
```

`Icons.navigate_next` is an `IconData`; `Icon` draws it. An icon-only button needs a description: wrap it in a `Tooltip`. `onPressed: () {}` is enabled but does nothing; `null` disables.

Salary app: the files

`main.dart`	starts the app, `home: Page1()`
`employee.dart`	model: valid hours and rate, the salary
`page1.dart`	controllers, checks input, pushes
`page2.dart`	reads the employee, shows the result

```
class Employee {
  final int hours;
  final double rate;
  static const double tax = 0.10;
  Employee(this.hours, this.rate) {
    if (hours <= 0 || !rate.isFinite
        || rate <= 0) {
      throw ArgumentError(
        'Hours and rate must be positive.');
    }
  }
  double netSalary() =>
      hours * rate * (1 - tax);
}
```

40 h at 25: gross 1000, net **900.00**. The 10% is an exercise rule.

Send an object: RouteSettings

```
// page1: check, build, push
final hours = int.tryParse(_hours.text.trim());
final rate =
    double.tryParse(_rate.text.trim());
if (hours == null || hours <= 0 ||
    rate == null || !rate.isFinite ||
    rate <= 0) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text(
      'Enter positive whole hours ...')));
  return;
}
Navigator.of(context).push<void>(
  MaterialPageRoute<void>(
    builder: (context) => const Page2(),
    settings: RouteSettings(
        arguments: Employee(hours, rate)),
  ));
```

```
// page2: read and check the type
final args =
    ModalRoute.of(context)?.settings.arguments;
Text(args is Employee
    ? args.toString()
    : 'No employee data was provided.')
```

- The route says *where*; the arguments say *what data*.
- `arguments` is `Object?`: check with `is`, which also promotes the type.
- Controllers: state fields, created once, disposed in `dispose`; never created in `build`.

Copyrighted content Dr Mohamad AOUDE

Better: a typed constructor

```
class SalaryPage extends StatelessWidget {
  const SalaryPage({super.key,
                    required this.employee});
  final Employee employee;
  ...
}
MaterialPageRoute<void>(builder: (context) =>
    SalaryPage(employee: employee))
```

The analyzer checks the type: no cast, no `MaterialPageRoute`. No `const`: the employee is a runtime value.

Named routes

```
MaterialApp(
  initialRoute: '/',
  routes: {'/': (context) =>
                const CategoriesPage()},
  onGenerateRoute: (settings) { ... },
  onUnknownRoute: (settings) => errorRoute,
)
Navigator.of(context).pushNamed<void>(
  CategoryPage.routeName,   // '/category'
  arguments: category);
```

- `static const routeName` avoids repeating the string.
- `routes` is a **Map** of name → builder.
- `home` already defines `'/'`: not both.
- A non-root `initialRoute` puts `'/'` beneath it. Not a login barrier.
- Flutter does not recommend named routes for new apps: use direct routes, or a Router (`go_router`) for web URLs and deep links.

onGenerateRoute: check the arguments

```
final argument = settings.arguments;
if (settings.name == CategoryPage.routeName
    && argument is String
    && ['Food', 'Drinks'].contains(argument)) {
  return MaterialPageRoute<void>(
    settings: settings,     // keep name + args
    builder: (context) =>
        CategoryPage(category: argument));
}
return null;  // -> onUnknownRoute
```

Filter, find, label

```
final entries = menuEntries
  .where((e) => e.category == widget.category)
  .toList();
final entry = menuEntries
  .firstWhere((e) => e.id == argument);
final kindText = switch (entry.kind) {
  ItemKind.meal  => 'Meal',
  ItemKind.drink => 'Drink',
};
```

`where`: all matches; `firstWhere`: one, **throws** if none (check first). An `enum` is a fixed set of typed values.

Return a result

```
Future<void> _openDetail(MenuEntry e) async {
  final result = await Navigator.of(context)
    .pushNamed<String>(DetailPage.routeName,
                       arguments: e.id);
  if (!mounted || result == null) return;
  setState(() => _chosen = result);
}
// on the detail screen:
Navigator.of(context).pop<String>(e.title);
```

Arguments go forward; the result comes back. The back arrow returns `null`: the previous choice stays.

Words to get right

`Route`	a navigation object, not a widget
`Navigator`	a widget; `.of(context)` gives its state
Arguments	stored in the route settings, not in the context
`{'id': ..}`	a Map; `List<Item>` is a list
`!`	asserts non-null at run time; not a check
Text themes	`bodyMedium`, `bodyLarge`, `titleLarge` (not `bodyText2`, `headline6`)

Common errors

Back needs many presses	you pushed instead of popping
Navigator not found	context above `MaterialApp`
Crash on `as Employee`	check with `is`, or a typed constructor
Input lost on rebuild	controllers created in `build`
Error only in the console	show a `SnackBar` or a validator
“Must be positive” for 0	match the message to the condition

Copyrighted content Dr Mohamad AOUDE